

LAB SESSION 01

Array Implementation and Traversal

THEORY

Arrays are a fundamental data structure in computer science used to store elements of the same type. They are stored in contiguous memory locations and accessed using indices. Arrays provide constant time access to elements, making them efficient for various applications such as storing lists, buffers, or matrices.

Advantages of arrays include ease of access, and disadvantages include fixed size and costly insertions/deletions. Arrays are used in sorting, searching, and implementing other data structures like heaps or hash tables.

PROCEDURE

Step-by-step instructions to implement the data structure.

1. The program starts by including a library that allows it to show messages on the screen.
2. It sets up the environment so you don't need to write extra words like "std::" before standard commands.
3. The main part of the program begins.
4. An array is created that can hold 5 numbers. These numbers are: 1, 2, 3, 4, and 5.
5. A loop is used to go through each number in the array one by one.
6. For each number in the array, the program prints:
7. The position of the number (starting from 0),
8. And the value at that position.
9. After printing all the elements, the program ends.

CODE

```
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for(int i = 0; i < 5; i++) {
        cout << "Element " << i << ": " << arr[i] << endl;
    }
}
```

```
    }  
    return 0;  
}
```

EXPECTED OUTPUT

Element at index 0 = 10
Element at index 1 = 20
Element at index 2 = 30
Element at index 3 = 40
Element at index 4 = 50

EXERCISES

1. Modify the program to input values from the user instead of hardcoded values.
2. Write a function to calculate the **sum** and **average** of all array elements.
3. Find the **maximum** and **minimum** value in the array.
4. Display the array elements in **reverse order**.
5. Write a function to find the **second largest element** in the array.
6. Write a function that calculates the **frequency of each unique element** in the array

